

final_service 문제 해결 및 변경 전·후 비교 보고서

항목	내용
작업일	2026-07-13 (Asia/Seoul)
대상	C:\wpknu_202601\final_service
서비스	React frontend / Spring Boot backend / Flask AI backend
범위	진단에서 확인된 API 연동, 설정 보안, CORS, 운영 서버, React 경고 및 필터 동작 수정
데이터 안전	검증용 민원 INSERT는 수행하지 않음. 읽기 API와 빈 요청 유효성 검사만 수행

결론

세 서비스가 모두 구동되며 React와 Flask 사이의 실제 민원 접수 경로가 연결되었다. React 빌드는 소스 경고 없이 성공했고, Spring 테스트 및 Oracle 연동 API, Flask CORS·유효성 검사, Waitress 운영 모드가 모두 통과했다.

1. 문제별 원본 대비 해결 결과

문제	변경 전(원본)	변경 후	상태
민원 신청 미연동	PetitionApply가 API를 호출하지 않고 localStorage에만 임시 저장	axios로 Flask /api/complaints 호출. 로그인 accountId 또는 비회원 이름·전화번호 전달	해결
API 목적지 불일치	프론트 proxy는 8080(Spring)뿐이고 Flask는 5001	REACT_APP_COMPLAINT_API_URL=http://localhost:5001을 사용해 AI 접수만 Flask로 명시적 라우팅	해결
이중 제출 가능	AI 처리 중에도 제출 버튼 활성화	isSubmitting 상태와 버튼 비활성화, 진행 문구 표시	해결
Flask 설정 기본값 노출	DB URL·계정과 개발 secret이 Python 코드 기본값	SECRET_KEY/DATABASE_URL 필수 검사. 실제 값은 gitignore 대상 .env로 분리	해결
과도한 CORS	모든 Origin(*) 허용	CORS_ORIGINS 목록만 허용. localhost:3000 허용 및 임의 Origin 헤더 미발급 확인	해결
개발 서버 운영 사용	항상 Flask 내장 서버로 실행	개발 모드만 Flask 서버, 그 외 환경은 Waitress WSGI 사용	해결
React Hook/미사용 경고	빌드 시 다수 exhaustive-deps/no-unused-vars 경고	의존성 정정, useCallback 안정화, 미사용 상태 제거 및 loading UI 활용	해결
반송 기간 필터 무동작	filterPeriod 값은 변경되지만 목록 필터에 미사용	최근 1주/1개월 cutoff 날짜 필터 적용	해결
Node 호환 범위 불명확	Node 24에서도 CRA 5를 실행해 내부 deprecated 경고 발생	package.json engines에 지원 범위 Node 18 이상 23 미만 명시	완화/명시

2. 파일 단위 수정·추가·삭제 내역

수정 파일

파일	원본	수정 내용
frontend/src/pages/PetitionApply.js	localStorage 모의 접수	Flask API 접수, 오류/422 처리, 계정 검증, 중복 제출 방지
frontend/.env	Host 검사 설정만 존재	REACT_APP_COMPLAINT_API_URL 추가
frontend/package.json	Node 지원 범위 미표기	engines.node >=18 <23 추가
frontend/src/pages/Notice.js	Effect 의존성 누락	navigate/selectedNotice 의존성 반영
frontend/src/pages/OfficerNotice.js	Effect 의존성 누락	navigate/selectedNotice 의존성 반영
frontend/src/pages/OfficerPetitionDetail.js	authHeader 함수가 매 렌더마다 재생성	useCallback 및 정확한 의존성 적용
frontend/src/pages/OfficerPetitions.js	미사용 petitions, 불안정 함수 의존성	상태 제거, useCallback 적용, 검색 인자 명시
frontend/src/pages/PetitionEdit.js	미사용 isLoggedIn 상태	미사용 상태와 설정 제거
frontend/src/pages/ReturnedPetitionDetail.js	Effect 의존성 누락	auth/fetch 함수를 useCallback으로 안정화
frontend/src/pages/ReturnedPetitions.js	loading 미표시, 기간 필터 미동작	로딩 행 표시, 기간 필터 구현, Hook 안정화
backend_flask/complaint_backend/_init_.py	하드코딩 fallback, CORS *	필수 환경변수 검사, 안전한 테스트 fallback, Origin 제한
backend_flask/app.py	항상 개발 서버	환경에 따라 Flask/Waitress 분기
backend_flask/requirements.txt	운영 WSGI 없음	waitress==3.0.2 추가
backend_flask/.env.example	CORS 설정 없음	CORS_ORIGINS 예시 추가
backend_flask/README.md	개발 실행 설명만 존재	개발/운영 서버 및 프론트 주소 설정 설명 추가

추가 파일

파일	목적
backend_flask/.env	로컬 실행 설정. 보고서에는 비밀번호와 secret을 기록하지 않음
backend_flask/.gitignore	.env, __pycache__, *.pyc 버전관리 제외
generate_change_report.py	본 PDF를 동일 형식으로 다시 생성하는 스크립트
final_service_문제해결_변경전후_보고서.pdf	최종 변경 비교 및 검증 보고서

삭제 파일

삭제된 파일은 없다. 기존 기능 파일은 보존하고 필요한 코드만 수정했다.

3. 주요 코드 흐름 비교

구분	변경 전	변경 후
민원 신청	Form → localStorage → 성공 alert	Form → Flask 5001 → AI 필터/표준화 → Oracle 저장 → 실제 응답 alert
회원 식별	loginId를 로컬 임시 데이터에 기록	로그인 응답에서 저장된 accountId를 USER_INFO 참조 userId로 전송
비회원 식별	브라우저 세션 전화번호와 이름을 로컬에 저장	이름-전화번호를 Flask에 보내 GUEST/COMPLAINT 관계로 저장
실패 처리	서버 실패 개념 없음	400/422/503/500 응답 메시지를 구분해 사용자에게 표시
Flask 시작	app.run(debug 설정)	development: app.run / production: waitress.serve
CORS	모든 사이트가 API 응답을 읽을 수 있음	설정된 프론트 Origin만 Access-Control-Allow-Origin 발급

4. 검증 결과

검증 항목	결과	증거
React production build	통과	Compiled successfully, ESLint 소스 경고 0건
React development server	통과	localhost:3000 HTTP 200
Spring Gradle test	통과	BUILD SUCCESSFUL, 4 actionable tasks
Spring service	통과	8080 /api/dashboard/today HTTP 200
Spring Oracle 조회	통과	8080 /api/complaints HTTP 200
Flask health	통과	5001 /api/health HTTP 200
Flask 유효성 검사	통과	빈 POST /api/complaints → HTTP 400 (DB 변경 없음)
허용 CORS	통과	Origin localhost:3000 → 허용 헤더 반환
비허용 CORS	통과	Origin evil.example → 허용 헤더 없음
Waitress 운영 모드	통과	production, 5002 /api/health HTTP 200
Oracle 직접 연결	통과	Oracle 21.3.0.0.0 연결 확인
Ollama/Gemma	통과	11434 응답 및 gemma:latest 설치 확인

검증 제한 및 주의

실제 정상 민원 POST는 Oracle에 영구 데이터가 생성되므로 검증 과정에서는 수행하지 않았다. 대신 같은 라우트의 유효성 검사, CORS, 모델/Ollama 준비 상태, Oracle 연결을 각각 확인했다. 첨부파일 UI는 기존에도 파일명만 보관하며 서버 업로드 API가 없으므로 이번 접수 연동 범위에는 포함하지 않았다.

5. 실행 방법

서비스	명령	주소
Spring	backend#gradlew.bat bootRun	http://localhost:8080
Flask 개발	backend_flask에서 python app.py	http://localhost:5001
Flask 운영	FLASK_ENV=production 후 python app.py	http://localhost:5001 (Waitress)
React	frontend에서 npm start	http://localhost:3000

운영 전 확인사항

- 1) backend_flask/.env의 SECRET_KEY와 DB 비밀번호를 운영용 secret으로 교체한다.
- 2) frontend의 REACT_APP_COMPLAINT_API_URL을 실제 Flask 배포 주소로 바꾼 뒤 다시 빌드한다.
- 3) CORS_ORIGINS에는 실제 프론트 도메인만 등록한다.
- 4) CRA 5 호환을 위해 Node 18~22 LTS를 사용한다.
- 5) 실제 접수 통합시험은 테스트 DB 또는 삭제 가능한 테스트 계정으로 수행한다.